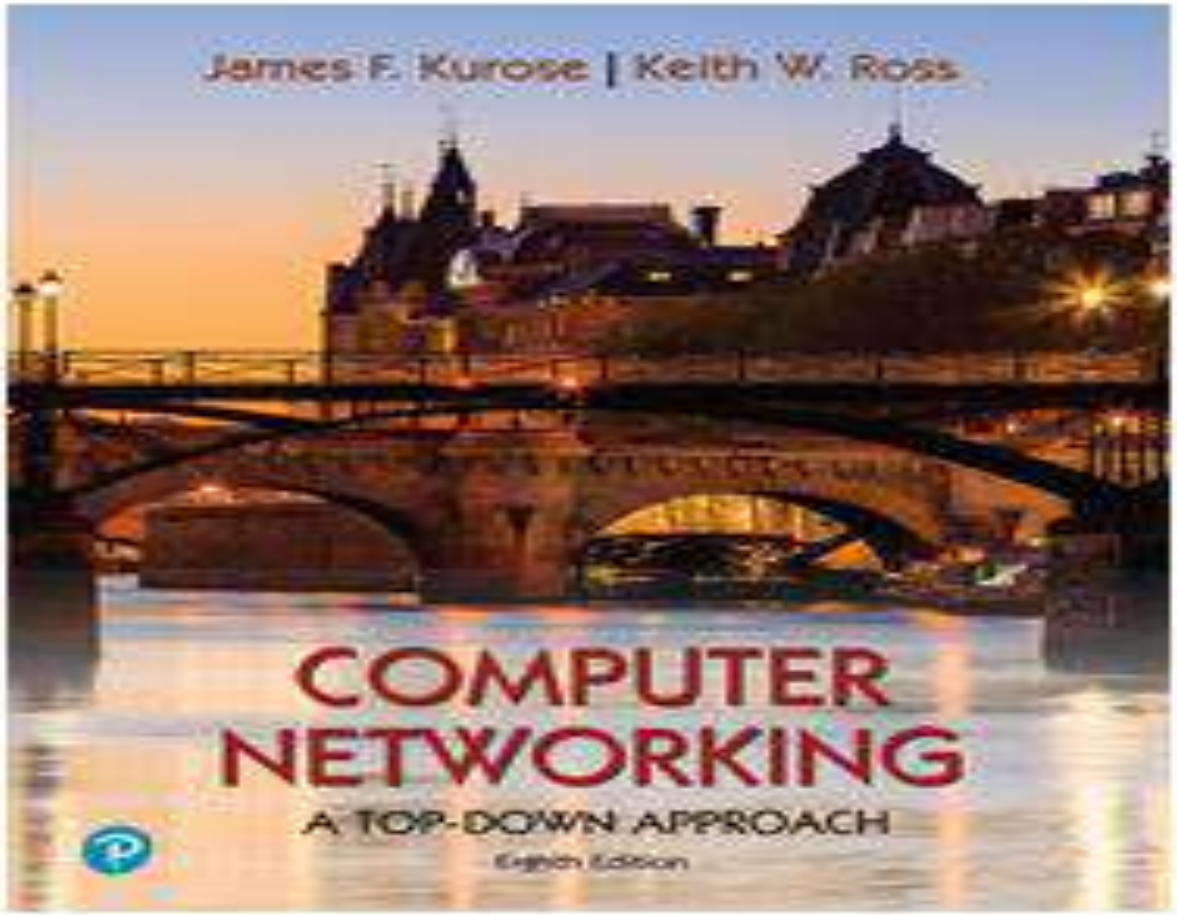


Computer Networking: A Top-Down Approach

8th edition
Jim Kurose, Keith Ross
Pearson, 2020

Chapter 2 Application Layer

Since the Internet's inception, numerous useful and entertaining applications have indeed been created.
These applications have been the driving force behind the Internet's success, motivating people in homes, schools, governments, and businesses to make the Internet an integral part of their daily activities.

Internet applications include the classic text-based applications that became popular in the 1970s and 1980s: text e-mail, remote access to computers, file transfers, and newsgroups. They include *the* killer application of the mid-1990s, the World Wide Web, encompassing Web surfing, search, and electronic commerce.

Since the beginning of new millennium, new and highly compelling applications continue to emerge, including voice over IP and video conferencing such as Skype, Facetime, and Google Hangouts; user generated video such as YouTube and movies on demand such as Netflix; and multiplayer online games such as Second Life and World of Warcraft.

During this same period, we have seen the emergence of a new generation of social networking applications—such as Facebook, Instagram, and Twitter—which have created human networks on top of the Internet's network or routers and communication links.

And most recently, along with the arrival of the smartphone and the ubiquity of 4G/5G wireless Internet access, there has been a profusion of location based mobile apps, including popular check-in, dating, and road-traffic forecasting apps (such as Yelp, Tinder, and Waz), mobile payment apps (such as WeChat and Apple Pay) and messaging apps (such as WeChat and WhatsApp).
Clearly, there has been no slowing down of new and exciting Internet applications.

Perhaps some of the readers of this text will create the next generation of killer Internet applications!

In this chapter, we study the conceptual and implementation aspects of network applications.
We begin by defining key application-layer concepts, including network services required by applications, clients and servers, processes, and transport-layer interfaces.

We examine several network applications in detail, including the Web, e-mail, DNS, peer-to-peer (P2P) file distribution, and video streaming.
We then cover network application development, over both TCP and UDP.
In particular, we study the socket interface

2.1 Principles of Network Applications

At the core of network application development is writing programs that run on different end systems and communicate with each other over the network.

For example, in the Web application there are two distinct programs that communicate with each other: the browser program running in the user's host (desktop, laptop, tablet, smartphone, and so on); and the Web server program running in the Web server host.

As another example, in a Video on Demand application such as Netflix, there is a Netflix-provided program running on the user's smartphone, tablet, or computer; and a Netflix server program running on the Netflix server host.

Servers often (but certainly not always) are housed in a data center,
Thus, when developing your new application, you need to write software that will run on multiple end systems.

This software could be written, for example, in C, Java, or Python.
Importantly, you do not need to write software that runs on network-core devices, such as routers or link-layer switches.
Even if you wanted to write application software for these network-core devices, you wouldn't be able to do so.

This basic design—namely, confining application software to the end systems, has facilitated the rapid development and deployment of a vast array of network applications.

2.1.1 Network Application Architectures

Keep in mind that an application's architecture is distinctly different from the network architecture (e.g., the five-layer Internet architecture discussed in Chapter 1).

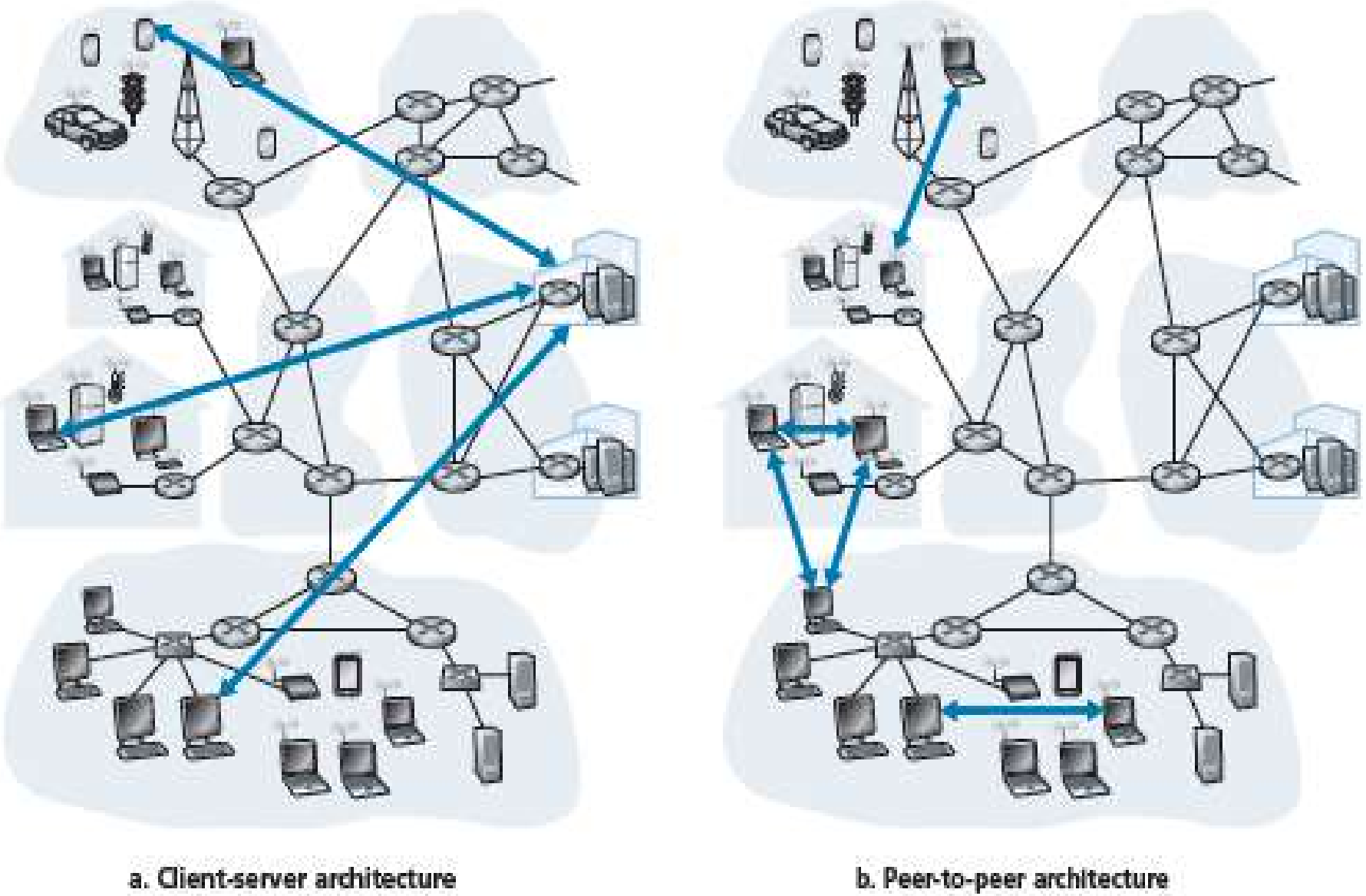
From the application developer's perspective, the network architecture is fixed and provides a specific set of services to applications.

The **application architecture**, on the other hand, is designed by the application developer and dictates how the application is structured over the various end systems.
In choosing the application architecture, an application developer will likely draw on one of the two predominant architectural paradigms used in modern network applications: the client-server architecture or the peer-to-peer (P2P) architecture.

In a **client-server architecture**, there is an always-on host, called the *server*, which services requests from many other hosts, called *clients*.
A classic example is the Web application for which an always-on Web server services requests from browsers running on client hosts.
When a Web server receives a request for an object from a client host, it responds by sending the requested object to the client host.

Note that with the client-server architecture, clients do not directly communicate with each other; for example, in the Web application, two browsers do not directly communicate.

Another characteristic of the client-server architecture is that the server has a fixed, well-known address, called an IP address (which we'll discuss soon).
Because the server has a fixed, well-known address, and because the server is always on, a client can always contact the server by sending a packet to the server's IP address.
Some of the better-known applications with a client-server architecture include the Web, FTP, Telnet, and e-mail.
The client-server architecture is shown in Figure 2.2(a).



a. Client-server architecture

b. Peer-to-peer architecture

Figure 2.2 ♦ (a) Client-server architecture; (b) P2P architecture

Often in a client-server application, a single-server host is incapable of keeping up with all the requests from clients.

For example, a popular social-networking site can quickly become overwhelmed if it has only one server handling all of its requests.

For this reason, a **data center**, housing a large number of hosts, is often used to create a powerful virtual server.
The most popular Internet services—such as search engines (e.g., Google, Bing, Baidu), Internet commerce (e.g., Amazon, eBay, Alibaba), Web-based e-mail (e.g., Gmail and Yahoo Mail), social media (e.g., Facebook, Instagram, Twitter, and WeChat)—run in one or more data centers.

In a **P2P architecture**, there is minimal (or no) reliance on dedicated servers in data centers. Instead the application exploits direct communication between pairs of intermittently connected hosts, called *peers*.
The peers are not owned by the service provider, but are instead desktops and laptops controlled by users, with most of the peers residing in homes, universities, and offices. Because the peers communicate without passing through a dedicated server, the architecture is called peer-to-peer.

An example of a popular P2P application is the file-sharing application BitTorrent.

One of the most compelling features of P2P architectures is their **self-scalability**.
For example, in a P2P file-sharing application, although each peer generates workload by requesting files, each peer also adds service capacity to the system by distributing files to other peers.

P2P architectures are also cost effective, since they normally don't require significant server infrastructure and server bandwidth (in contrast with clients-server designs with datacenters).

However, P2P applications face challenges of security, performance, and reliability due to their highly decentralized structure.

2.1.2 Processes Communicating

Before building your network application, you also need a basic understanding of how the programs, running in multiple end systems, communicate with each other.

In the jargon of operating systems, it is not actually programs but **processes** that communicate.

A process can be thought of as a program that is running within an end system.
When processes are running on the same end system, they can communicate with each other with interprocess communication, using rules that are governed by the end system's operating system.
But in this book, we are not particularly interested in how processes in the same host communicate, but instead in how processes running on *different* hosts (with potentially different operating systems) communicate.

Processes on two different end systems communicate with each other by exchanging **messages** across the computer network.
A sending process creates and sends messages into the network; a receiving process receives these messages and possibly responds by sending messages back.

processes communicating with each other reside in the application layer of the five-layer protocol stack.

Client and Server Processes

A network application consists of pairs of processes that send messages to each other over a network.
For example, in the Web application a client browser process exchanges messages with a Web server process.

In a P2P file-sharing system, a file is transferred from a process in one peer to a process in another peer.

For each pair of communicating processes, we typically label one of the two processes as the **client** and the other process as the **server**.

With the Web, a browser is a client process and a Web server is a server process.

With P2P file sharing, the peer that is downloading the file is labeled as the client, and the peer that is uploading the file is labeled as the server.
You may have observed that in some applications, such as in P2P file sharing, a process can be both a client and a server.
Indeed, a process in a P2P file-sharing system can both upload and download files.
Nevertheless, in the context of any given communication session between a pair of processes, we can still label one process as the client and the other process as the server.

We define the client and server processes as follows:

In the context of a communication session between a pair of processes, the process that initiates the communication (that is, initially contacts the other process at the beginning of the session) is labeled as the client*.*
The process that waits to be contacted to begin the session is the server*.*

In the Web, a browser process initializes contact with a Web server process; hence the browser process is the client and the Web server process is the server.

In P2P file sharing, when Peer A asks Peer B to send a specific file, Peer A is the client and Peer B is the server in the context of this specific communication session.
When there's no confusion, we'll sometimes also use the terminology "client side and server side of an application."

The Interface Between the Process and the Computer Network

As noted above, most applications consist of pairs of communicating processes, with the two processes in each pair sending messages to each other.
Any message sent from one process to another must go through the underlying network.

A process sends messages into, and receives messages from, the network through a software interface called a **socket**.

Let's consider an analogy to help us understand processes and sockets.
A process is analogous to a house and its socket is analogous to its door.
When a process wants to send a message to another process on another host, it shoves the message out its door (socket).

This sending process assumes that there is a transportation infrastructure on the other side of its door that will transport the message to the door of the destination process..

Once the message arrives at the destination host, the message passes through the receiving process's door (socket), and the receiving process then acts on the message.

Figure 2.3 illustrates socket communication between two processes that communicate over the Internet.
(Figure 2.3 assumes that the underlying transport protocol used by the processes is the Internet's TCP protocol.)

As shown in this figure, a socket is the interface between the application layer and the transport layer within a host.

It is also referred to as the **Application Programming Interface (API)** between the application and the network, since the socket is the programming interface with which network applications are built.

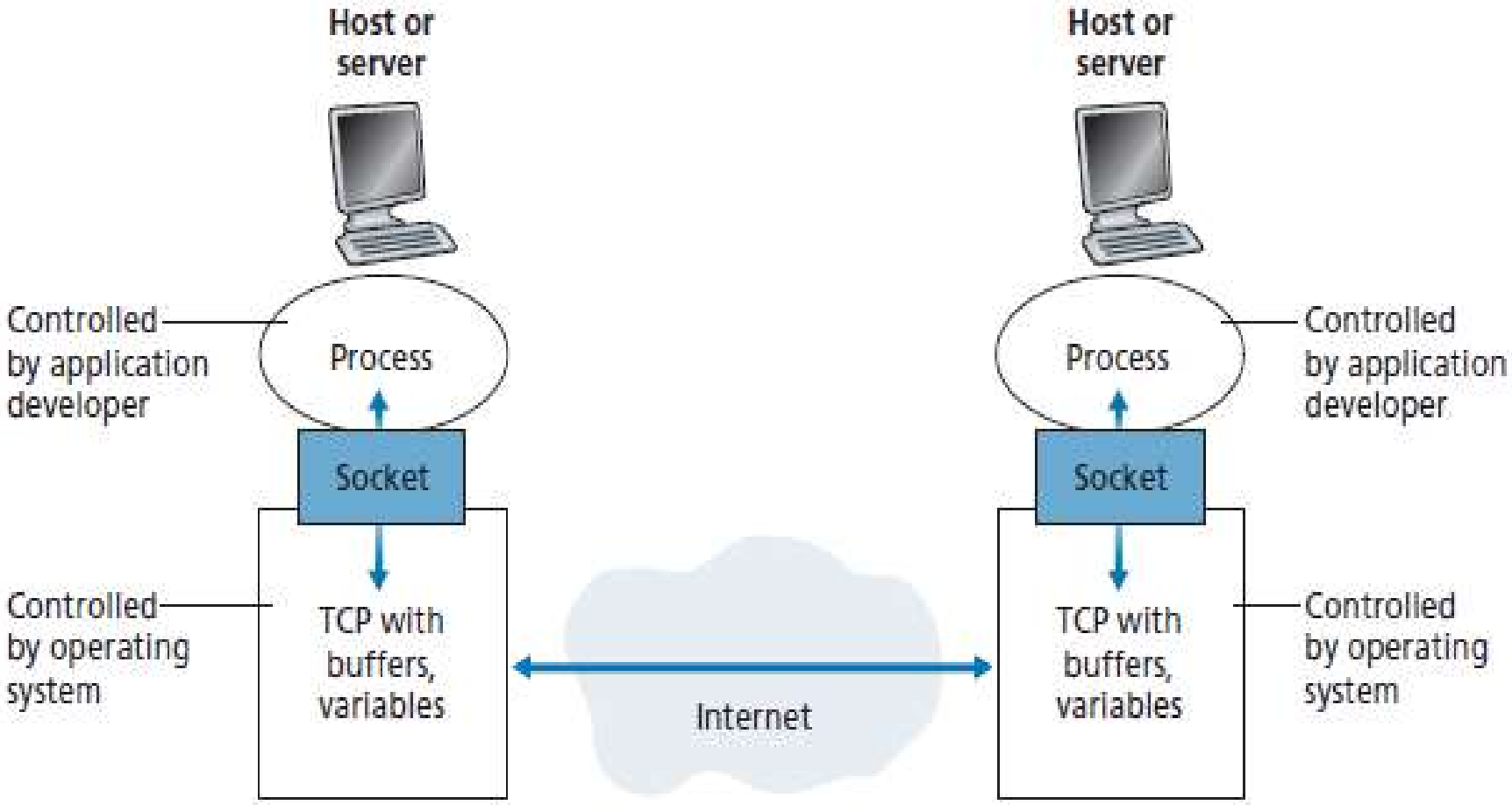


Figure 2.3 ♦ Application processes, sockets, and underlying transport protocol

The application developer has control of everything on the application-layer side of the socket but has little control of the transport-layer side of the socket.

The only control that the application developer has on the transport-layer side is
(1) the choice of transport protocol and
(2) perhaps the ability to fix a few transport-layer parameters such as maximum buffer and maximum segment sizes (to be covered in Chapter 3).

Once the application developer chooses a transport protocol (if a choice is available), the application is built using the transport-layer services provided by that protocol.

Addressing Processes

In order to send postal mail to a particular destination, the destination needs to have an address. Similarly, in order for a process running on one host to send packets to a process running on another host, the receiving process needs to have an address.

To identify the receiving process, two pieces of information need to be specified:
(1) the address of the host and
(2) an identifier that specifies the receiving process in the destination host.

In the Internet, the host is identified by its **IP address**.
We'll discuss IP addresses in great detail in Chapter 4.
For now, all we need to know is that an IP address is a 32-bit quantity that we can think of as uniquely identifying the host.

In addition to knowing the address of the host to which a message is destined, the sending process must also identify the receiving process (more specifically, the receiving socket) running in the host. This information is needed because in general a host could be running many network applications.
A destination **port number** serves this purpose.

Popular applications have been assigned specific port numbers.
For example, a Web server is identified by port number 80.
A mail server process (using the SMTP protocol) is identified by port number 25.

A list of well-known port numbers for all Internet standard protocols can be found at www.iana.org.

2.1.3 Transport Services Available to Applications

Recall that a socket is the interface between the application process and the transport-layer protocol.
The application at the sending side pushes messages through the socket.
At the other side of the socket, the transport-layer protocol has the responsibility of getting the messages to the socket of the receiving process.

Many networks, including the Internet, provide more than one transport-layer protocol.
When you develop an application, you must choose one of the available transport-layer protocols. How do you make this choice? Most likely, you would study the services provided by the available transport-layer protocols, and then pick the protocol with the services that best match your application's needs.

What are the services that a transport-layer protocol can offer to applications invoking it? We can broadly classify the possible services along four dimensions: reliable data transfer, throughput, timing, and security.

Reliable Data Transfer

As discussed in Chapter 1, packets can get lost within a computer network.
For example, a packet can overflow a buffer in a router, or can be discarded by a host or router after having some of its bits corrupted.

For many applications—such as electronic mail, file transfer, remote host access, Web document transfers, and financial applications—data loss can have devastating consequences (in the latter case, for either the bank or the customer!).
Thus, to support these applications, something has to be done to guarantee that the data sent by one end of the application is delivered correctly and completely to the other end of the application.
If a protocol provides such a guaranteed data delivery service, it is said to provide **reliable data transfer**.

One important service that a transport-layer protocol can potentially provide to an application is process-to-process reliable data transfer.
When a transport protocol provides this service, the sending process can just pass its data into the socket and know with complete confidence that the data will arrive without errors at the receiving process.

When a transport-layer protocol doesn't provide reliable data transfer, some of the data sent by the sending process may never arrive at the receiving process.
This may be acceptable for **loss-tolerant applications**, most notably multimedia applications such as conversational audio/video that can tolerate some amount of data loss.

Throughput

In Chapter 1, we introduced the concept of available throughput, which, in the context of a communication session between two processes along a network path, is the rate at which the sending process can deliver bits to the receiving process.

Because other sessions will be sharing the bandwidth along the network path, and because these other sessions will be coming and going, the available throughput can fluctuate with time.

These observations lead to another natural service that a transport-layer protocol could provide, namely, guaranteed available throughput at some specified rate.

With such a service, the application could request a guaranteed throughput of r bits/sec, and the transport protocol would then ensure that the available throughput is always at least r bits/sec.

Such a guaranteed throughput service would appeal to many applications.

For example, if an Internet telephony application encodes voice at 32 kbps, it needs to send data into the network and have data delivered to the receiving application at this rate.
If the transport protocol cannot provide this throughput, the application would need to encode at a lower rate (and receive enough throughput to sustain this lower coding rate) or may have to give up, since receiving, say, half of the needed throughput is of little or no use to this Internet telephony application.

Applications that have throughput requirements are said to be **bandwidth-sensitive applications**. Many current multimedia applications are bandwidth sensitive, although some multimedia applications may use adaptive coding techniques to encode digitized voice or video at a rate that matches the currently available throughput.

While bandwidth-sensitive applications have specific throughput requirements, **elastic applications** can make use of as much, or as little, throughput as happens to be available. Electronic mail, file transfer, and Web transfers are all elastic applications.
Of course, the more throughput, the better.

Timing

A transport-layer protocol can also provide timing guarantees.

As with throughput guarantees, timing guarantees can come in many shapes and forms.
An example guarantee might be that every bit that the sender pumps into the socket arrives at the receiver's socket no more than 100 msec later.
Such a service would be appealing to interactive real-time applications, such as Internet telephony, virtual environments, teleconferencing, and multiplayer games, all of which require tight timing constraints on data delivery in order to be effective, see [Gauthier 1999; Ramjee 1994].

Long delays in Internet telephony, for example, tend to result in unnatural pauses in the conversation; in a multiplayer game or virtual interactive environment, a long delay between taking an action and seeing the response from the environment (for example, from another player at the end of an end-to-end connection) makes the application feel less realistic.

For non-real-time applications, lower delay is always preferable to higher delay, but no tight constraint is placed on the end-to-end delays.

Security

Finally, a transport protocol can provide an application with one or more security services.

For example, in the sending host, a transport protocol can encrypt all data transmitted by the sending process, and in the receiving host, the transport-layer protocol can decrypt the data before delivering the data to the receiving process.
Such a service would provide confidentiality between the two processes, even if the data is somehow observed between sending and receiving processes.

A transport protocol can also provide other security services in addition to confidentiality, including data integrity and end-point authentication, topics that we'll cover in detail in Chapter 8.